رویداد نخ مسدودی که منتظر آن بود اتفاق می‌افتد، هسته ندای دیگری به کتابخانه‌ی نخ می‌فرستد تا اطلاع بدهد که هم‌اکنون نخ مسدود پیشین می‌تواند اجرا شود. راه‌انداز ندای این رویداد نیز به یک پردازنده‌ی مجازی نیاز خواهد داشت که هسته می‌تواند پردازنده‌ی مجازی جدیدی تخصیص بدهد یا یکی از نخ‌های کاربر را پس‌گرفته و راه‌انداز ندا را بر روی پردازنده‌ی مجازی آن اجرا کند. بعد از این که، نخ از حالت انسداد خارج شد و برای آمادگی اجرا علامت خورد، برنامه‌ی کاربردی، نخ آماده را به منظور اجرا بر روی یکی از پردازنده‌های مجازی در دسترس زمانبندی می‌نماید.

۴-۷ مثال‌های سیستم‌عاملی

تا اینجا، یک سری مفهوم و ملاحظات مربوط به نخ‌ها را بررسی کردیم. این فصل را با مرور چگونگی پیاده‌سازی نخ‌ها در سیستم‌های ویندوز و لینوکس جمع می‌بندیم.

۴-۷-۱ نخ‌های ویندوز

یک برنامه‌ی کاربردی ویندوز به صورت پردازه‌ای مجزا اجرا می‌شود و هر پردازه می‌تواند یک یا چندین نخ را شامل شود. API ویندوز برای ایجاد نخ‌ها در بخش ۴-۴-۲ پوشش داده شد. به علاوه، ویندوز از نگاشت یک-به-یک مطرح شده در بخش ۴-۳-۲ استفاده می‌کند که هر نخ سطح کاربر به نخی از هسته نگاشته می‌شود.

مؤلفه‌های کلی یک نخ عبارتند از:

- یک ID نخ که نخ را به صورت یکتا مشخص می‌کند.
- یک مجموعه ثبات که وضعیت پردازنده را نشان می‌دهد.
- شمارنده‌ی برنامه
- پشته‌ی کاربر، که موقع اجرای نخ در مد کاربر به‌کار می‌رود، و پشته‌ی هسته، که موقع اجرای نخ در مد هسته کاربرد دارد.
- یک ناحیه‌ی حافظه‌ی خصوصی که توسط کتابخانه‌های مختلف زمان اجرا و کتابخانه‌های اتصالی پویا[1] (DLL) استفاده می‌شود.

مجموعه ثبات، پشته‌ها و ناحیه‌ی خصوصی حافظه به **متن**[2] نخ معروفند.

ساختمان‌داده‌های اصلی یک نخ عبارتند از:

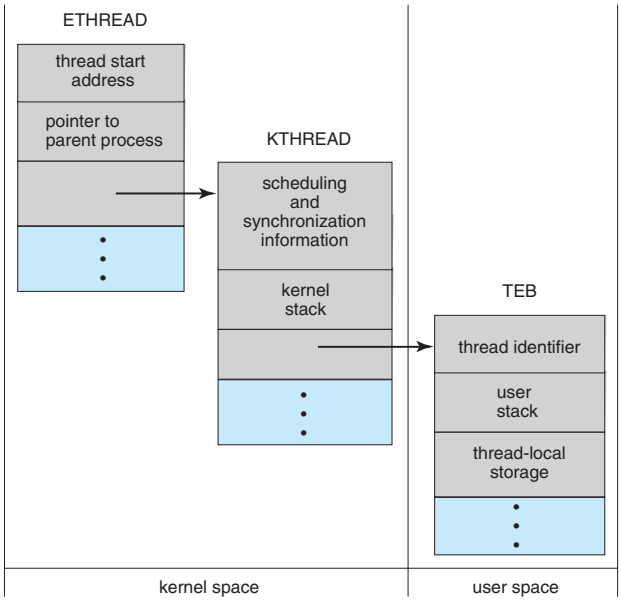
- ETHREAD□بلاک اجرایی نخ
- KTHREAD□بلاک نخ هسته
- TEB□بلاک محیط نخ

مؤلفه‌های کلیدی ETHREAD شامل اشاره‌گری به پردازه‌ی تعلق نخ و آدرس روالی است که نخ در آن کنترل را به‌دست می‌گیرد. همچنین ETHREAD، یک اشاره‌گر به KTHREAD متناظر دارد.

1. Dynamic link libraries
2. Context

اطلاعات مربوط به زمانبندی و همگامی نخ در KTHREAD قرار دارد. به علاوه، KTHREAD شامل پشته‌ی هسته (که موقع اجرای نخ در مد هسته کاربرد دارد) و یک اشاره‌گر به TEB می‌باشد.

بلاک‌های ETHREAD و KTHREAD کلاً در فضای هسته قرار دارند؛ این بدان معنی است که تنها هسته می‌تواند آنها را دستیابی کند. TEB یک ساختمان‌داده‌ی فضای کاربر است که موقع اجرای نخ در مد کاربر دستیابی می‌شود. TEB در کنار سایر فیلدها، شناسه‌ی نخ، پشته‌ی مد کاربر و آرایه‌ای برای حافظه‌ی محلی نخ را در بر دارد. ساختار یک نخ ویندوز در شکل ۴-۲۱ نشان داده می‌شود.



شکل ۴-۲۱ ساختمان‌داده‌های یک نخ ویندوز.

۴-۷-۲ نخ‌های لینوکس

لینوکس، فراخوانی سیستمی ()fork را با کارکرد سابق تکثیر پردازه، چنان که در فصل ۳ مطرح شد، فراهم می‌کند. لینوکس قابلیت ایجاد نخ‌ها را با استفاده از فراخوانی سیستمی ()clone نیز فراهم می‌نماید. با این وجود، لینوکس تفاوتی میان پردازه‌ها و نخ‌ها قایل نمی‌شود. در حقیقت، لینوکس در زمان رجوع به یک روند کنترل در یک برنامه، از اصطلاح ***وظیفه***—به جای ***پردازه*** یا ***نخ***—استفاده می‌کند.

زمانی که ()clone احضار می‌شود، یک مجموعه پرچم که میزان اشتراک میان وظایف والد و فرزند را معین می‌کنند، به آن ارسال می‌شود. فهرست برخی از این پرچم‌ها در شکل ۴-۲۲ لیست شده‌اند. برای مثال، فرض کنید پرچم‌های CLONE_FS، CLONE_VM، CLONE_SIGHAND و CLONE_FILES به تابع ()clone ارسال شوند. در ادامه، وظیفه‌های والد و فرزند، اطلاعات

سیستم‌فایل یکسان (از قبیل دایرکتوری کاری فعلی)، فضای حافظه‌ی یکسان، راه‌اندازهای سیگنال یکسان و مجموعه فایل باز یکسان را به اشتراک می‌گیرند. استفاده از ()clone با این شیوه معادل ایجاد نخی است که در این فصل مطرح شد، چون وظیفه‌ی والد بیشتر منابع خود را با وظیفه‌ی فرزندش به اشتراک می‌گذارد. با این وجود، اگر موقع احضار ()clone، هیچ یک از این پرچم‌ها انتخاب نشوند، هیچ‌گونه اشتراکی صورت نمی‌گیرد که نتیجه‌ای مشابه کارکرد فراخوانی سیستمی ()fork را خواهد داشت.

به خاطر شیوه‌ی نمایش یک وظیفه در هسته‌ی لینوکس، سطوح اشتراک مختلفی امکان‌پذیر است. برای هر وظیفه در سیستم، یک ساختمان‌داده‌ی یکتا در هسته (به ویژه، struct task_struct) وجود دارد. این ساختمان‌داده، به جای ذخیره‌سازی داده‌های وظیفه، شامل اشاره‌گرهایی به سایر ساختمان‌داده‌هایی است که این داده‌ها در آنها ذخیره شده‌اند—برای مثال، ساختمان‌داده‌هایی که لیست فایل‌های باز، اطلاعات راه‌اندازی سیگنال و حافظه‌ی مجازی را نشان می‌دهند. زمانی‌که ()fork احضار می‌شود، وظیفه‌ی جدیدی به همراه یک **کپی** از تمامی ساختمان‌داده‌های مربوط به پردازه‌ی والد ایجاد می‌شود. در زمان انجام فراخوانی سیستمی ()clone نیز یک وظیفه‌ی جدید ایجاد می‌شود. با این وجود، به جای این که تمامی ساختمان‌داده‌ها کپی شوند، بسته به مجموعه‌ی پرچم‌هایی که به ()clone ارسال می‌شوند، وظیفه‌ی جدید به ساختمان‌داده‌هایی از وظیفه‌ی والد ***اشاره می‌کند***.

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.

شکل ۴-۲۲ برخی از پرچم‌هایی که موقع احضار ()clone ارسال می‌شوند.

در نهایت، انعطاف‌پذیری فراخوانی سیستمی ()clone می‌تواند به مفهوم ظرف‌ها[1] تعمیم یابد، یک موضوع مجازی‌سازی که در فصل ۱ معرفی شد. از همان فصل به خاطر بیاورید که ظرف یک تکنیک مجازی‌سازی است که توسط سیستم‌عامل فراهم می‌شود و امکان ایجاد سیستم‌های لینوکس متعدد (ظرف‌ها) را تحت یک هسته‌ی لینوکس منفرد که به‌صورت مجزا از یک‌دیگر اجرا می‌شوند ممکن می‌سازد. درست مثل پرچم‌های خاص ارسالی به ()clone که می‌توان بر اساس میزان اشتراک میان وظایف والد و فرزند، بین ایجاد یک وظیفه که بیشتر شبیه به یک پردازه یا نخ رفتار می‌کند تمایز قایل شد، پرچم‌های دیگری هستند که می‌توان به ()clone ارسال کرد که امکان ایجاد یک ظرف لینوکس را ممکن می‌سازند. ظرف‌ها به‌صورت کامل در فصل ۱۸ پوشش داده خواهند شد.

1. Containers